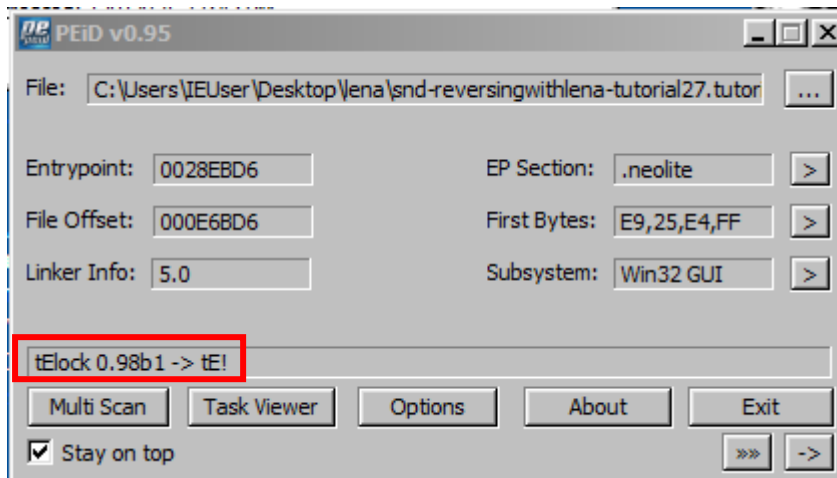


악성코드 분석 보고서

(sand-reversingwithlana-tutorials)

2025.11.26

1. Bootca.exe – 언패킹



tElock 0.98 패커를 사용하고 있다.

※ tElock이란?

2000~2001년경 나온 Windows용 실행파일 보호(Packer/Protector)

- PE 압축 / 암호화
- 예외 기반 흐름 제어 (SEH Redirection)
- 안티디버깅 + 리다이렉션 기능 포함

└ 안티디버깅 기능

+ CRC 체크 → 코드가 변했는지 검사

+ Hardware Breakpoint 감지

+ 중간 코드 변경 감시

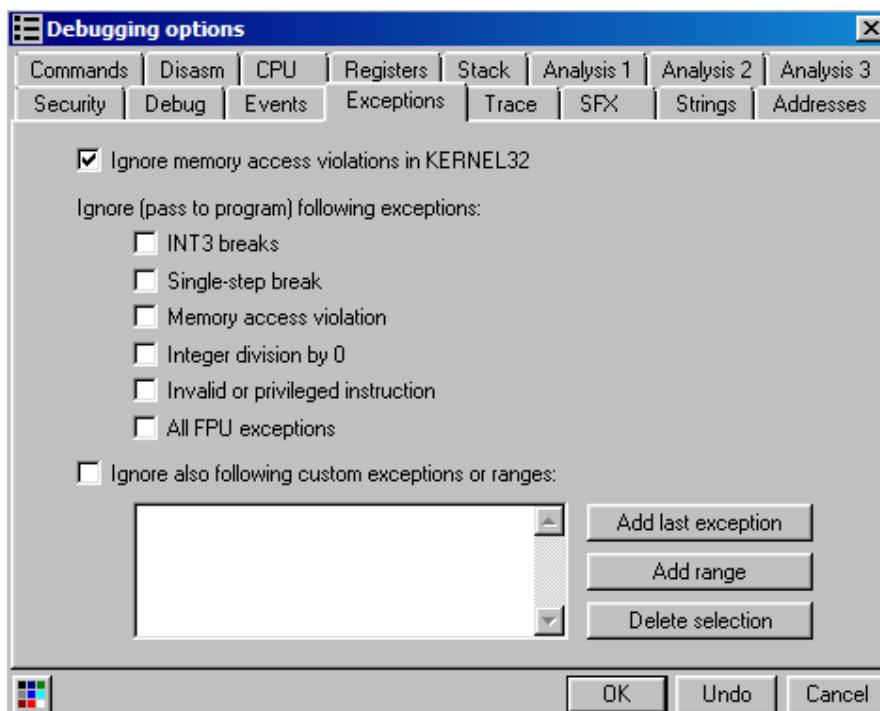
+ 디버거의 예외 처리 개입 여부 감지

└ 리다이렉션 기능

+ 원본 코드 주소를 바로 쓰지 않고:

- 점프 테이블을 여러 겹으로 감추거나
- SEH를 이용해 "엉뚱한 곳 → 실제 코드"로 넘기거나
- 자동 복구 루틴을 여러 번 사용하는 방식

패커	SEH 예외 기반?	Shift+F9 OEP 접근 가능?
tElock	예	가능
Armadillo(구버전)	예	가능
Themida/WinLicense 초기	일부	가능할 때도 있음
UPX	아님	불가능
ASPack	아님	불가능
FSG/MEW	아님	불가능
최신 Themida/VMProtect	아님(가상화)	절대 불가능



"tElock"는 CRC-Check, HW BP 탐지 등 다양한 안티 디버깅이 적용되어 있다. 따라서 이 프로텍터가 적용된 바이너리의 OEP를 찾기 위해 이전에 보여주었던 "Exceptions"을 이용할 것이다.

CPU - main thread, module Bootca		
0068EBD6	^ E9 25E4FFFF	JMP Bootca.0068D000
0068EBDB	0000	ADD BYTE PTR DS:[EAX],AL
0068EBDD	00F6	ADD DH,DH
0068EBDF	1E	PUSH DS
0068EBE0	^ 7F CD	JG SHORT Bootca.0068EBAF
0068EBE2	1E	PUSH DS
0068EBE3	EC	IN AL,DX
0068EBE4	2800	SUB BYTE PTR DS:[EAX],AL
0068EBE6	0000	ADD BYTE PTR DS:[EAX],AL
0068EBE8	0000	ADD BYTE PTR DS:[EAX],AL
0068EBEA	0000	ADD BYTE PTR DS:[EAX],AL
0068EBEC	0000	ADD BYTE PTR DS:[EAX],AL

올리디버그로 Bootca.exe를 실행시키면 위의 코드에서 시작된다.

tElock는 SHE 예외기반을 사용하기 때문에 'shift + f9'을 이용해서 OEP를 찾아준다.

* shift + f9: 예외가 발생했을 때, 예외 처리 루틴을 지나서 정상 실행 지점까지 한 번에 진행시키는 기능

0068E7C5	48	DEC EAX	Bootca
0068E7C6	BF 00000000	MOV EDI,	File Window ?
0068E7CB	64:8F07	POP DWOR	
0068E7CE	5F	POP EDI	
0068E7CF	EB 02	JMP SHOR	
0068E7D1	FF20	JMP DWOR	
0068E7D3	3D 007DCF4F	CMP EAX,	
0068E7D8	60	PUSHAD	
0068E7D9	E8 06000000	CALL Boc	
0068E7DE	8B6424 08	MOV ESP,	
0068E7E2	EB 1A	JMP SHOR	
0068E7E4	64:67:FF36 0000	PUSH DWOR	
0068E7EA	64:67:8926 0000	MOV DWOR	
0068E7F0	9C	PUSHFD	
0068E7F1	810C24 00010000	OR DWOR	
0068E7F8	9D	POPFD	
0068E7F9	F8	CLC	
0068E7FA	^ 73 DC	JNB SHOR	
0068E7FC	CD 20	INT 20	
0068E7FE	64:67:8F06 0000	POP DWOR	
0068E804	58	POP EAX	

21번 째에 프로그램이 실행되는 걸 볼 수 있다. 그럼 20번 째까지 간 다음에 OEP를 찾아줄거다.

0068E7D8	→ 60	PUSHAD
0068E7D9	E8 06000000	CALL Bootca.0068E7E4
0068E7DE	8B6424 08	MOV ESP,DWORD PTR SS:[ESP+8]
0068E7E2	EB 1A	JMP SHORT Bootca.0068E7FE
0068E7E4	64:67:FF36 0000	PUSH DWORD PTR FS:[0]
0068E7EA	64:67:8926 0000	MOV DWORD PTR FS:[0],ESP
0068E7F0	9C	PUSHFD
0068E7F1	810C24 00010000	OR DWORD PTR SS:[ESP],100
0068E7F8	9D	POPFD
0068E7F9	F8	CLC
0068E7FA	^ 73 DC	JNB SHORT Bootca.0068E7D8
0068E7FC	CD 20	INT 20

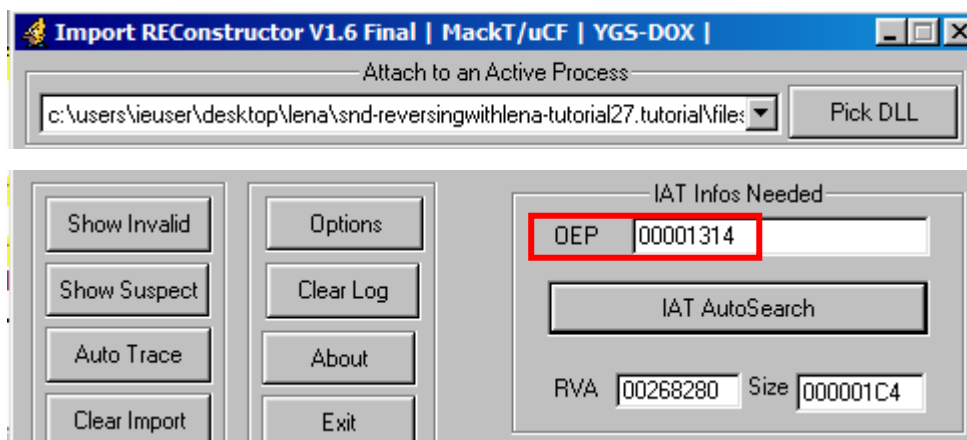
M Memory map								
Address	Size	Owner	Section	Contains	Type	Access	Initial	M
00250000	00008000				Priv	RW	RW	
00400000	00001000	Bootca		PE header	Imag	RW	RWE	
00401000	001E9000	Bootca	.neolite	code	Imag	RW	RWE	
005EA000	0007C000	Bootca	.neolite	data	Imag	RW	RWE	
006B6000	00001000	Bootca	.neolite		Imag	RW	RWE	
006B7000	00001000	Bootca	.neolite		Imag	RW	RWE	
006B8000	00003000	Bootca	.neolite		Imag	RW	RWE	

20번 째까지 간 후 Memory Map을 켜서 code에 BP를 걸어줄거다. 왜냐하면 OEP는 code에 위치하고 있기 때문이다. 그 상태에서 'shift + F9'을 한 번 더 눌러준다.

C CPU - main thread, module Bootca				
00401314	EB 10	JMP	SHORT Bootca.00401326	
00401316	66	DB	66	CHAR 'f'
00401317	62	DB	62	CHAR 'b'
00401318	3A	DB	3A	CHAR ':'
00401319	43	DB	43	CHAR 'C'
0040131A	2B	DB	2B	CHAR '+'
0040131B	2B	DB	2B	CHAR '+'
0040131C	48	DB	48	CHAR 'H'
0040131D	4F	DB	4F	CHAR 'O'
0040131E	4F	DB	4F	CHAR 'O'
0040131F	4B	DB	4B	CHAR 'K'
00401320	90	NOP		
00401321	E9	DB	E9	
00401322	98A05E00	DD	OFFSET Bootca.____CPPdebugHook	
00401326	> A1 8BA05E00	MOV	EAX, DWORD PTR DS: [5EA08B]	
0040132B	. C1E0 02	SHL	EAX, 2	
0040132E	. A3 8FA05E00	MOV	DWORD PTR DS: [5EA08F], EAX	
00401333	. 52	PUSH	EDX	
00401334	. 6A 00	PUSH	0	
00401336	. E8 8D7C1E00	CALL	Bootca.005E8FC8	
0040133B	. 8BD0	MOV	EDX, EAX	
0040133D	. E8 2E431B00	CALL	Bootca.005B5670	
00401342	. 5A	POP	EDX	
00401343	. E8 C4361B00	CALL	Bootca.005B4A0C	
00401348	. E8 27431B00	CALL	Bootca.005B5674	
0040134D	. 6A 00	PUSH	0	
0040134F	. E8 34651B00	CALL	Bootca.005B7888	
00401354	. 59	POP	ECX	

0x0041314에서 멈추는 걸 볼 수 있다. 이 부분이 OEP다.

여기서 덤프를 떠줘야한다고 생각하지만 tElock은 API주소를 리다이렉션하기 때문에 아직 덤프를 뜨면 안된다.



OEP를 1314(0041314 - 40000)로 바꿔준 후 IAT AutoSearch를 누르면 밑에 RVA랑 Size가 변환되서 나온다. Get Imports하기 전에 올리디버그로 해당 위치가 맞는지 확인해준다

Address	Hex dump
00668280	00 00 00 00 00 00 20 00 10 00 20 00 20 00 20 00
00668290	30 00 20 00 40 00 20 00 50 00 20 00 6E 00 20 00
006682A0	7F 00 20 00 90 00 20 00 9E 00 20 00 BE 00 20 00
006682B0	CF 00 20 00 E0 00 20 00 F1 00 20 00 00 01 20 00
006682C0	0E 01 20 00 1E 01 20 00 2E 01 20 00 3E 01 20 00
006682D0	4F 01 20 00 5F 01 20 00 6F 01 20 00 8F 01 20 00

Address	Hex dump
00668060	50 89 26 00 00 00 00 00 00 00 00 00 00 00 00 00
00668070	00 00 00 00 00 00 00 00 4B 8C 26 00 59 8C 26 00
00668080	6B 8C 26 00 7B 8C 26 00 8D 8C 26 00 9D 8C 26 00
00668090	B1 8C 26 00 C5 8C 26 00 00 00 00 00 ED 45 78 75
006680A0	B1 13 78 75 37 A8 79 75 6A A4 79 75 5B 48 78 75
006680B0	C3 E0 77 75 43 48 78 75 FB 13 78 75 00 00 00 00
006680C0	D7 8C 26 00 E5 8C 26 00 F3 8C 26 00 09 8D 26 00
006680D0	17 8D 26 00 25 8D 26 00 37 8D 26 00 45 8D 26 00

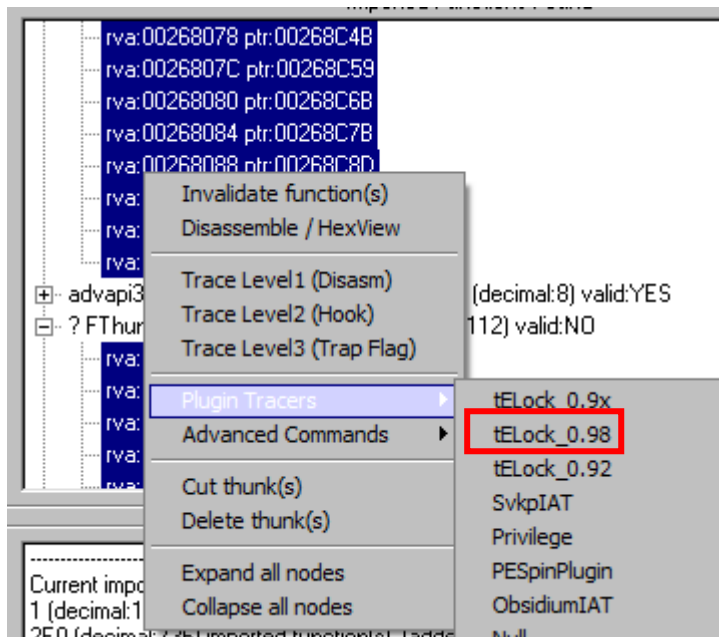
668280으로 가서 덤프 창을 보면 맞는거 같지만 스크롤을 위로 올려보면 위에 더 있는 걸 볼 수 있다. ImportREC가 잘못 가져왔으므로 바꿔줘야한다. **RVA: 0x00268074**

Address	Hex dump
00668BC0	EE 0A 23 00 FF 0A 23 00 0E 0B 23 00 1C 0B 23 00
00668BD0	2C 0B 23 00 3C 0B 23 00 4C 0B 23 00 5C 0B 23 00
00668BE0	6C 0B 23 00 7C 0B 23 00 9A 0B 23 00 AB 0B 23 00
00668BF0	BC 0B 23 00 CA 0B 23 00 EA 0B 23 00 FB 0B 23 00
00668C00	0C 0C 23 00 1D 0C 23 00 2C 0C 23 00 3A 0C 23 00
00668C10	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00668C20	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

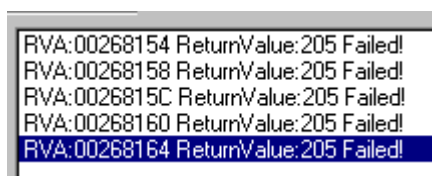
Size도 확인해보면 마지막 위치가 0x00668C10 처음 위치가 0x00668074라서 둘이 빼주면 사이즈가 0x00000BA0로 나와서 이것도 바꿔줘야한다. **Size: 0x00000BA0**

Imported Functions Found
⊕ ? FThunk:00268078 NbFunc:8 (decimal:8) valid:NO
⊕ advapi32.dll FThunk:0026809C NbFunc:8 (decimal:8) valid:YES
⊕ ? FThunk:002680C0 NbFunc:70 (decimal:112) valid:NO
⊕ ? FThunk:00268284 NbFunc:B7 (decimal:183) valid:NO
⊕ ? FThunk:00268564 NbFunc:48 (decimal:72) valid:NO
⊕ ? FThunk:00268688 NbFunc:B1 (decimal:177) valid:NO
⊕ ? FThunk:00268950 NbFunc:B0 (decimal:176) valid:NO

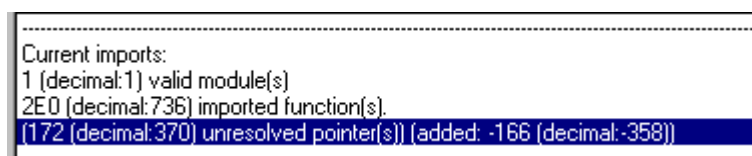
이렇게 많은 모듈이 valid:NO로 나오는 걸 볼 수 있다. 이럴 경우 플러그인을 이용 (tElock을 위한 플러그인 별도 존재)하면 문제가 되는 API 들을 찾고 해결하는 것이 굉장히 쉬워진다. 우선 플러그인이 어떤 행위를 하는지부터 볼 것이다.



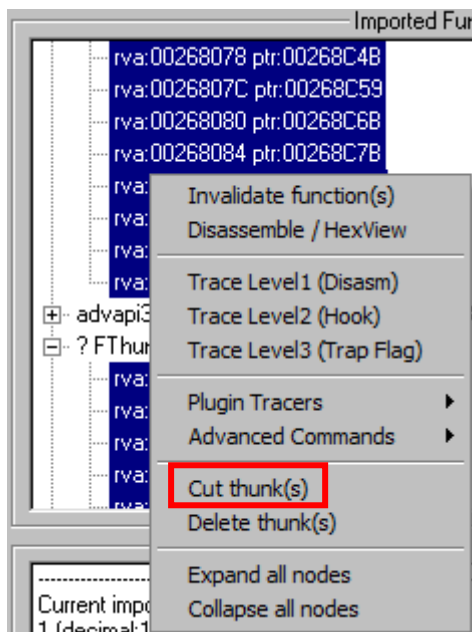
Show Invalid를 눌러서 Invalid된 걸 다 선택해준다. tELock 9.8버전이기 때문에 해당 플러그를 실행해준다.



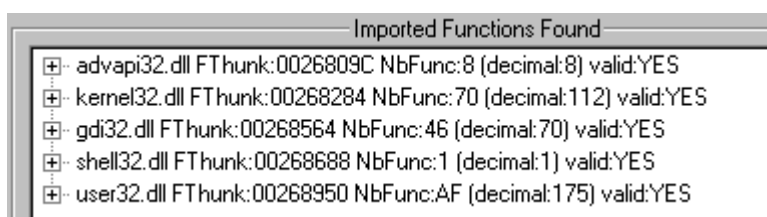
이런 식으로 돌아가는 걸 볼 수 있다. 그리고 tELock 프로텍터에서 인젝션한 API 함수도 있기 때문에 어떤 API는 "Return Value : 200 Found!" 라고 성공할 수도 있지만 어떤 API는 "Return Value : 205 Failed!" 라고 실패할 수도 있다.



다 돌아가면 해당 결과 값이 나온다.

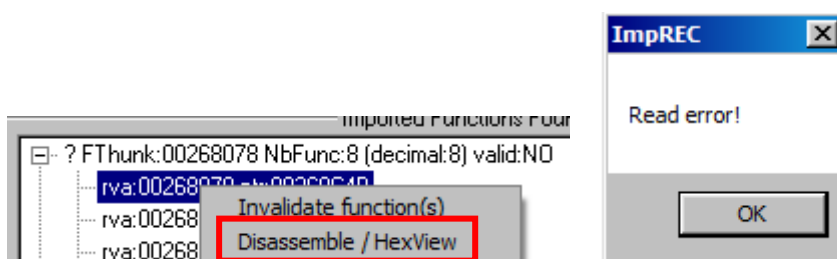


다시 Show Invalid를 눌러준 후 Cut thunk를 눌러준다. (플러그를 이용해서 API를 못찾은 함수들을 삭제하기 위해서다.)



그럼 이렇게 valid:YES로 나와서 IAT를 다 찾아준다. 이렇게 플러그인을 사용하는 것도 좋지만 손으로 풀어서 어떻게 돌아가는지 알아야한다.

다시 Clear Imports – Get Imports 눌러서 초기화를 해준다.



Valid:NO인 값들만 수동으로 확인해줄거다.

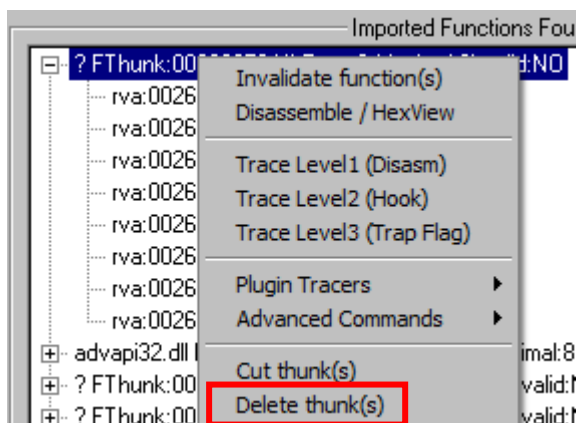
Disassemble / HexView를 눌러보면 Error가 발생하는 걸 볼 수 있다.

rva: 00268078 ptr:00268C4B 이렇게 있는데 rva는 IAT에서 해당 API의 위치이고 ptr은 해당 API의 코드의 위치를 뜻하고 있다.

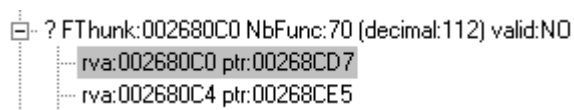
CPU - main thread, module Bootca			
00668C4B	0000	ADD BYTE PTR DS: [EAX], AL	
00668C4D	0000	ADD BYTE PTR DS: [EAX], AL	
00668C4F	0000	ADD BYTE PTR DS: [EAX], AL	
00668C51	0000	ADD BYTE PTR DS: [EAX], AL	
00668C53	0000	ADD BYTE PTR DS: [EAX], AL	
00668C55	0000	ADD BYTE PTR DS: [EAX], AL	
00668C57	0000	ADD BYTE PTR DS: [EAX], AL	
00668C59	0000	ADD BYTE PTR DS: [EAX], AL	
00668C5B	0000	ADD BYTE PTR DS: [EAX], AL	
00668C5D	0000	ADD BYTE PTR DS: [EAX], AL	
00668C5F	0000	ADD BYTE PTR DS: [EAX], AL	
00668C61	0000	ADD BYTE PTR DS: [EAX], AL	
00668C63	0000	ADD BYTE PTR DS: [EAX], AL	
00668C65	0000	ADD BYTE PTR DS: [EAX], AL	
00668C67	0000	ADD BYTE PTR DS: [EAX], AL	
00668C69	0000	ADD BYTE PTR DS: [EAX], AL	
00668C6B	0000	ADD BYTE PTR DS: [EAX], AL	

ImageBase + ptr을 해서 들어가면 이렇게 쓰레기 값이 있는 걸 볼 수 있다.

* ImportREC를 이용해서 에러가 나고 Lena랑 코드가 달라서 찾아보니 "tElock은 OS / 메모리배치 / SEH 루프 횟수에 따라 "중간 찌꺼기"가 미세하게 달라짐" 이런 이유 때문에 코드가 달라도 신경안써도 될거 같다.



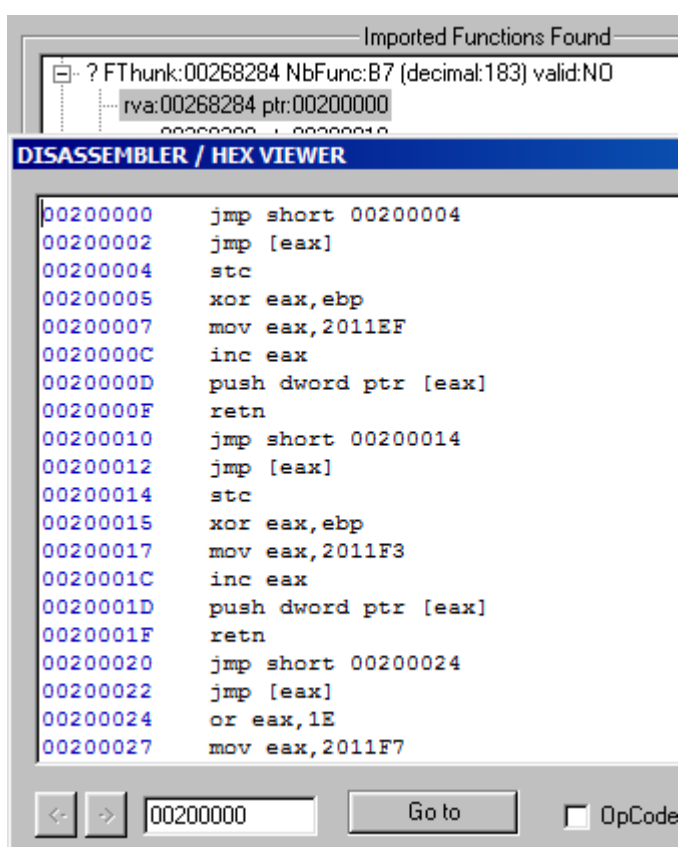
해당 부분은 쓰레기 값이므로 해당 모듈을 삭제해준다.



해당 모듈의 첫 번째 API도 Disassemble / HexView를 눌러보면 Error가 발생한다. 그래서 올리디버그로 확인해준다.

CPU - main thread, module Bootca				
00668CD7	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CD9	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CDB	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CDD	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CDF	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CE1	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CE3	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CE5	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CE7	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CE9	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CEB	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CED	0000	ADD	BYTE PTR DS:[EAX],AL	
00668CEE	0000	ADD	BYTE PTR DS:[EAX],AL	

이렇게 쓰레기 값이 있으므로 해당 모듈도 삭제해준다.



다음 모듈의 첫 번째 API는 Disassemble / HexView를 눌러보면 쓰레기 코드가 아니라 실행되는 코드가 나오는 걸 볼 수 있다.

1. "00200004" 주소로 점프
2. STC = "Carry Flag" 세팅 <= 잉여 명령어
3. "XOR EAX,EBP" 실행
4. "mov eax,2011EF" <= 이 명령어 때문에 위 "XOR EAX,EBP"는 잉여 명령어가 됨

5. "inc eax" <= EAX = 2011F0

6. "push eax" <= EAX 레지스터에 저장

7. "ret" <= " **PUSH + RET == JUMP, JMP EAX = JMP 2011F0** "

쉽게 말해서 실제 명령어는 "0x00200007"부터 시작인 것이다. 그러나 위 7가지 명령어는 결국 해석하면 "JMP 2011F0 "과 마찬가지다.

Address	Hex dump
002011D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
002011E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
002011F0	88 EA 81 76 81 EC 81 76 66 9E 81 76 DA 45 81 76
00201200	47 19 81 76 A8 DA 81 76 52 BE 82 76 D9 37 81 76
00201210	35 C0 81 76 1D C0 81 76 4C 4E 82 76 AB C1 81 76
00201220	69 A8 81 76 CD A6 81 76 05 A6 81 76 3F 87 80 76
00201230	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

덤프창에서 0x002011F0로 가보면 이렇게 해당 모듈의 모든 API 들의 원본 주소(76 XX XX XX) 들이 있는 걸 볼 수 있다.

00401322 98A0
00401326 > A1 8
0040132B - C1E0
0040132E - A3 8
00401333 - 52
00401334 - 6A 0
00401336 - E8 8
0040133B - 8BD0
0040133D - E8 2
00401342 - 5A
00401343 - E8 C
00401348 - E8 2
0040134D - 6A 0
0040134F - E8 3
00401354 - 59

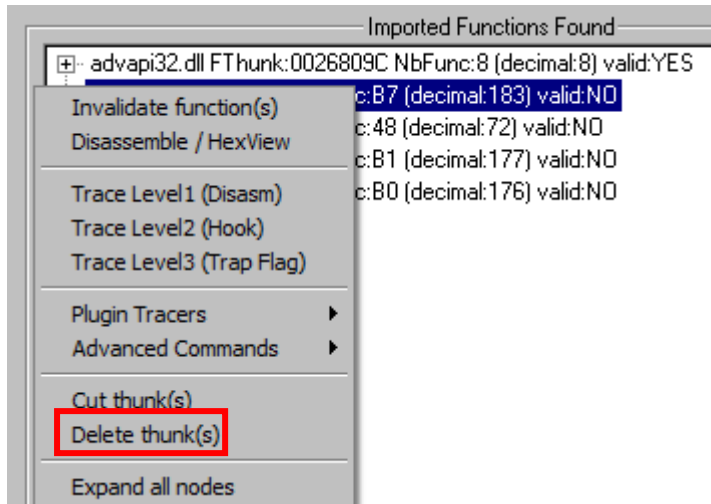
Backup
Copy
Binary
Breakpoint
Search for
Follow DWORD in Disassembler
Follow DWORD in Dump
Go to
Hex
Text
Short
Long
Float
Disassemble
Special
Data Ripper
Memory Dump
Appearance

Edit Ctrl+E
Fill with 00's
Fill with FF's
Binary copy
Binary paste

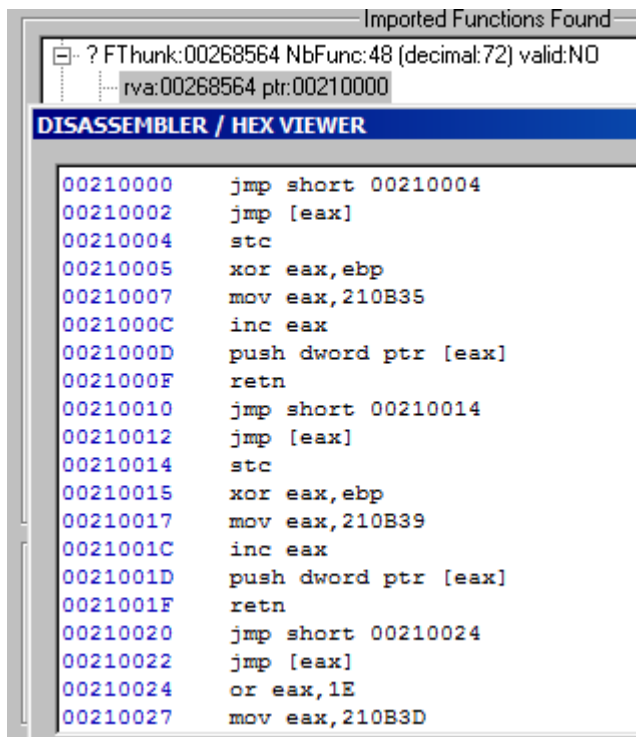
a.005B4A0C
a.005B5674
a.005B7888

Address	Hex dump
00201370	C1 08 83 F9 81 76 5A C6 81 76
00201380	3D 6D 82 F1 81 76 36 F5 85 76
00201390	16 56 82 93 80 76 2D 2D 81 76
002013A0	0F A8 81 76 F9 90 80 76 8A A2 81 76 32 5F 83 76
002013B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
002013C0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

0x002011F0에서부터 0x002013B3까지 복사를 해준다.



헛갈리지 않게 수정했으면 지워준다.



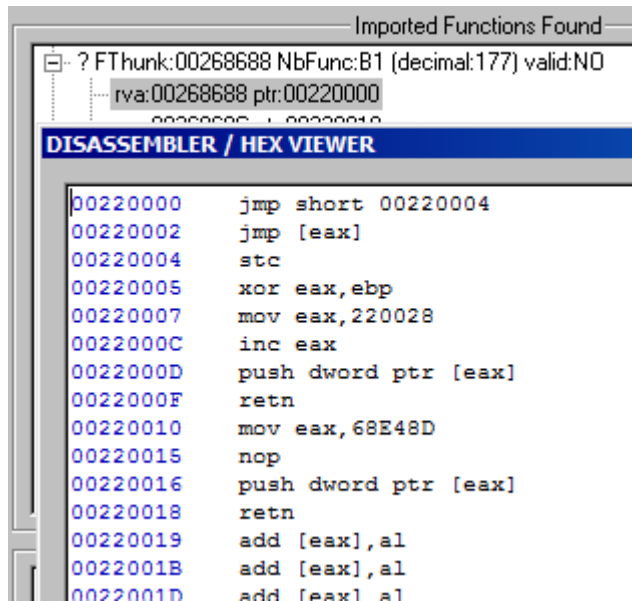
이것도 똑같이 0x00210B36으로 가서 덤프창을 확인해준다.

Address	Hex dump
00210B16	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00210B26	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00210B36	C0 72 F4 76 1E 65 F4 76 79 6B F4 76 AD 73 F4 76
00210B46	88 68 F4 76 CD 9B F4 76 D4 32 F5 76 B8 A5 F4 76
00210B56	2D 50 F7 76 1C D6 F4 76 31 D5 F4 76 B0 B3 F4 76
00210B66	CE F3 F4 76 3B 63 F4 76 FD 64 F4 76 49 6B F4 76

IAT가 리다이렉트된 걸 볼 수 있다 해당 부분 끝까지 (00 00 00 00) 복사하기를 해준다.

00668564	C0	72	F4	76	1E	65	F4	76	79	6B	F4	76	AD	73	F4	76
00668574	88	68	F4	76	CD	9B	F4	76	D4	32	F5	76	B8	A5	F4	76
00668584	2D	50	F7	76	1C	D6	F4	76	31	D5	F4	76	B0	B3	F4	76
00668594	CE	F3	F4	76	3B	63	F4	76	FD	64	F4	76	49	6B	F4	76
006685A4	C4	8B	F4	76	AA	6E	F4	76	14	5F	F4	76	7B	55	F7	76
006685B4	F7	FC	F5	76	E8	08	F6	76	FE	0A	F5	76	F9	FF	F4	76
006685C4	F5	01	F4	76	3D	75	F4	76	01	77	F4	76	75	65	F4	76

해당 모듈 IAT 시작 부분(0x00668564)에서 붙여넣기를 해주고 ImportREC로 돌아가서 헷갈리지 않게 해당 모듈을 삭제해준다.



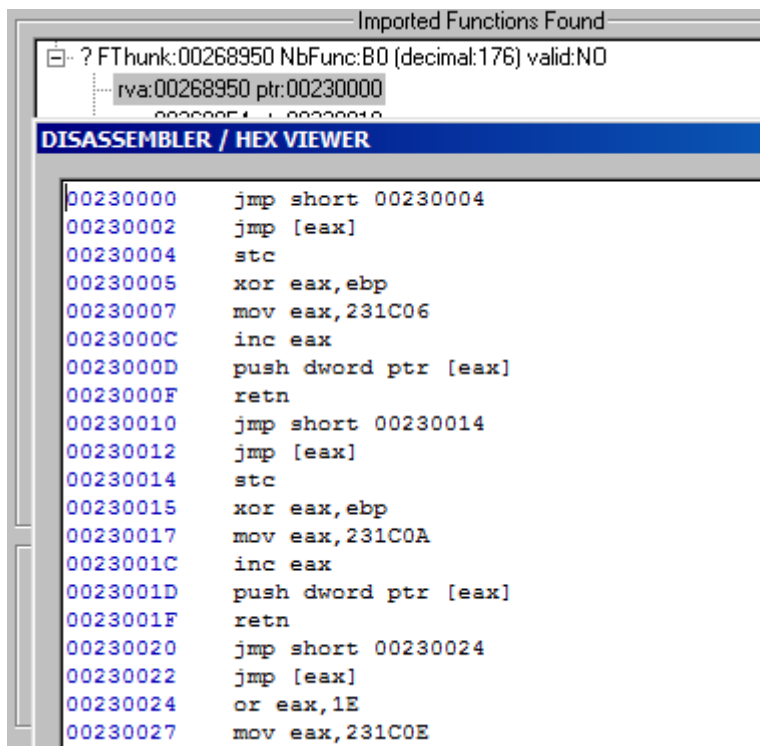
이것도 똑같이 0x00220029으로 가서 덤프창을 확인해준다.

Address	Hex dump
00220009	00 22 00 40 FF 30 C3 B8 8D E4 68 00 90 FF 30 C3
00220019	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00220029	90 87 BD 75 00 00 00 00 00 00 00 00 00 00 00
00220039	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

IAT가 리다이렉트된 걸 볼 수 있다 해당 부분 끝까지 (00 00 00 00) 복사하기를 해준다.

00668688	90	87	BD	75	00	00	00	00	B1	98	26	00	C7	98	26	00
00668698	D5	98	26	00	EB	98	26	00	F9	98	26	00	0D	99	26	00
006686A8	1F	99	26	00	31	99	26	00	3D	99	26	00	4B	99	26	00
006686B8	5D	99	26	00	65	99	26	00	75	99	26	00	87	99	26	00

해당 모듈 IAT 시작 부분(0x00668688)에서 붙여넣기를 해주고 ImportREC로 돌아가서 헷갈리지 않게 해당 모듈을 삭제해준다.



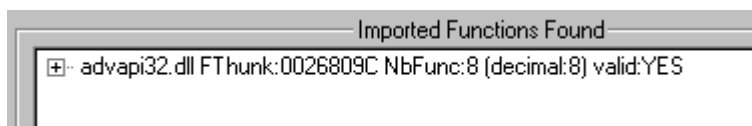
이것도 똑같이 0x00231C06으로 가서 덤프창을 확인해준다.

Address	Hex dump
00231BE6	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00231BF6	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00231C06	00 8A 48 8C 75 2B 3B 91 75 6E A6 8B 75 E4 5C 8C
00231C16	75 DB 03 8E 75 A3 2B 8E 75 0A 3B 8C 75 29 C8 8B

IAT가 리다이렉트된 걸 볼 수 있다 해당 부분 끝까지 (00 00 00 00) 복사하기를 해준다.

Address	Hex dump
00668930	DF A3 26 00 F3 A3 26 00 03 A4 26 00 11 A4 26 00
00668940	1D A4 26 00 2F A4 26 00 3B A4 26 00 00 00 00 00
00668950	00 8A 48 8C 75 2B 3B 91 75 6E A6 8B 75 E4 5C 8C
00668960	75 DB 03 8E 75 A3 2B 8E 75 0A 3B 8C 75 29 C8 8B
00668970	75 6F 98 8B 75 BF EC 8C 75 CE EB 8D 75 4C EE 8D
00668980	75 AA B6 8F 75 E6 12 8C 75 3C 44 8E 75 E9 40 8E

해당 모듈 IAT 시작 부분(0x00668950)에서 붙여넣기를 해주고 ImportREC로 돌아가서 헛갈리지 않게 해당 모듈을 삭제해준다.



마지막으로 이렇게 1개만 남은 걸 볼 수 있다.

00401319	43	DB 43
0040131A	2B	DB 2B
0040131B	2B	DB 2B
0040131C	48	DB 48
0040131D	4F	DB 4F
0040131E	4F	DB 4F
0040131F	4B	DB 4B
00401320	90	NOP
00401321	E9	DB E9
00401322	98A05E00	DD OFFSET B0
00401326	> A1 8BA05E00	MOV EAX,DWORD PTR [EAX+8BA05E00]
0040132B	. C1E0 02	SHL EAX,2
0040132E	. A3 8FA05E00	MOV DWORD PTR [EAX+8FA05E00],EAX
00401333	. 52	PUSH EDX
00401334	. 6A 00	PUSH 0
00401336	. E8 8D7C1E00	CALL Bootca
0040133B	. 8BD0	MOV EDX,EAX
0040133D	. E8 2E431B00	CALL Bootca
00401342	. 5A	POP EDX
00401343	. E8 C4361B00	CALL Bootca
00401348	. E8 27431B00	CALL Bootca
0040134D	. 6A 00	PUSH 0
0040134F	. E8 34651B00	CALL Bootca
00401354	. 59	POP ECX

Address	Hex dump
00668930	DF A3 26 00 F3 A3 26 00 03 A4 26 00
00668940	1D A4 26 00 2F A4 26 00 3B A4 26 00
00668950	00 8A 48 8C 75 2B 3B 91 75 68 8C 75 2B 3B 91 75 68

Backup
 Copy
 Binary
 Assemble Space
 Label :
 Comment ;
 Breakpoint
 Hit trace
 Run trace
 New origin here Ctrl+Gray *
 Go to
 Follow in Dump
 Search for
 Find references to
 View
 Copy to executable
 Analysis
 Detach Process
 Process Patcher
 Code Ripper
 Data Ripper
 Dump debugged process
 Ultra String Reference
 Appearance

올리디버그로 파일을 다 수정했으니까 이때 덤프를 떠야한다.

OllyDump - Bootca.exe

Start Address: Size: Dump

Entry Point: -> Modify: Get EIP as OEP Cancel

Base of Code: Base of Data:

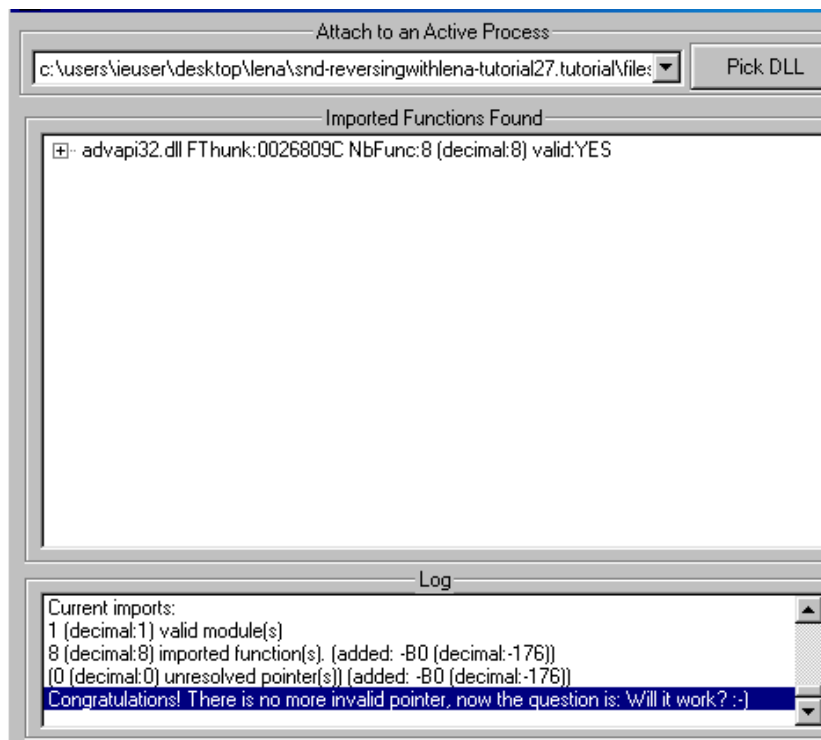
☒ Fix Raw Size & Offset of Dump Image

Section	Virtual Size	Virtual Offset	Raw Size	Raw Offset	Characteristics
.neolite	001E9000	00001000	001E9000	00001000	C0000040
.neolite	0007C000	001EA000	0007C000	001EA000	C0000040
.neolite	00001000	00266000	00001000	00266000	C0000040
.neolite	00001000	00267000	00001000	00267000	C0000040
.neolite	00003000	00268000	00003000	00268000	C0000040
.neolite	00001000	0026B000	00001000	0026B000	C0000040
.rsrc	00021000	0026C000	00021000	0026C000	C0000040
.neolite	00003000	0028D000	00003000	0028D000	C0000040

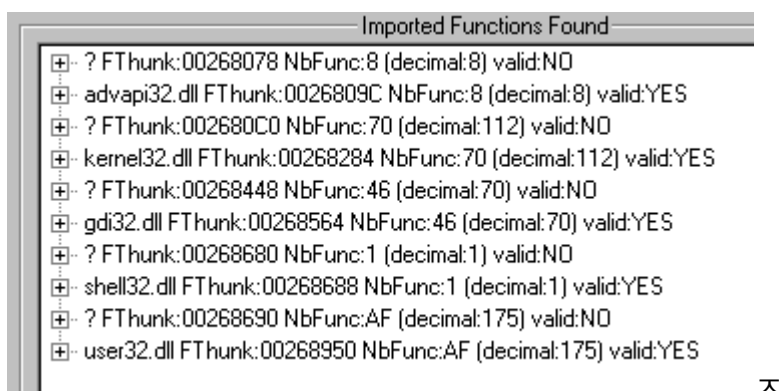
☐ Rebuild Import

☒ Method1 : Search JMP[API] | CALL[API] in memory image
☐ Method2 : Search DLL & API name string in dumped file

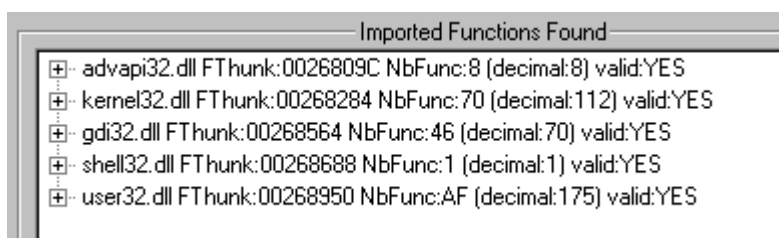
Rebuild Import를 체크 해제하고 덤프를 눌러준다.



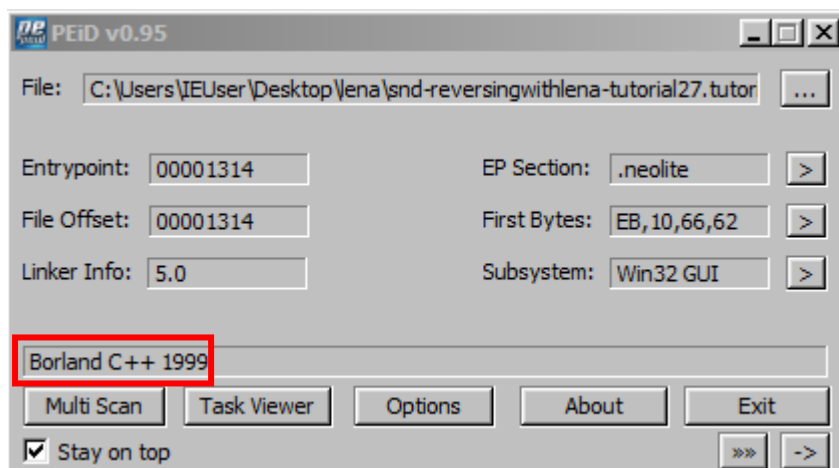
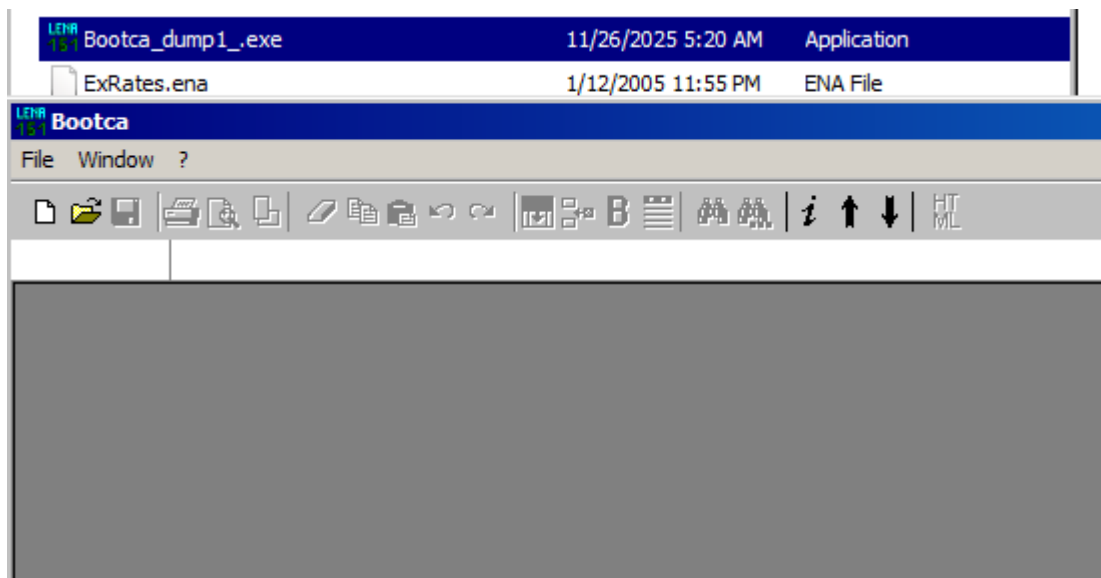
이 상태에서 Get Import를 한 번 눌러주면 올리디버그가 수정된 상태에서 IAT를 가져온다.



이렇게 나오는 걸 볼 수 있고 Show Invalid 해서 valid:NO된 부분을 다 없애준다.

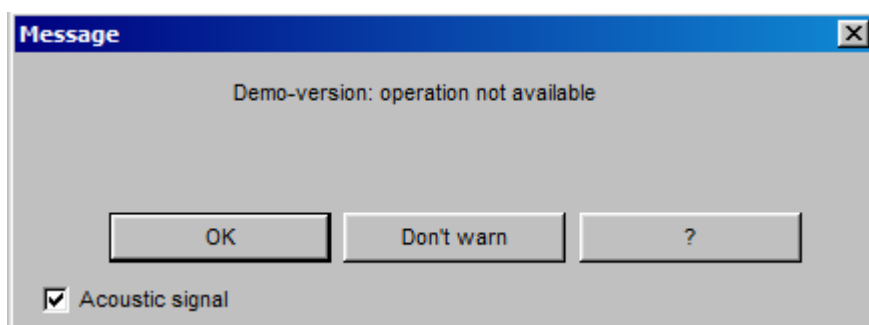
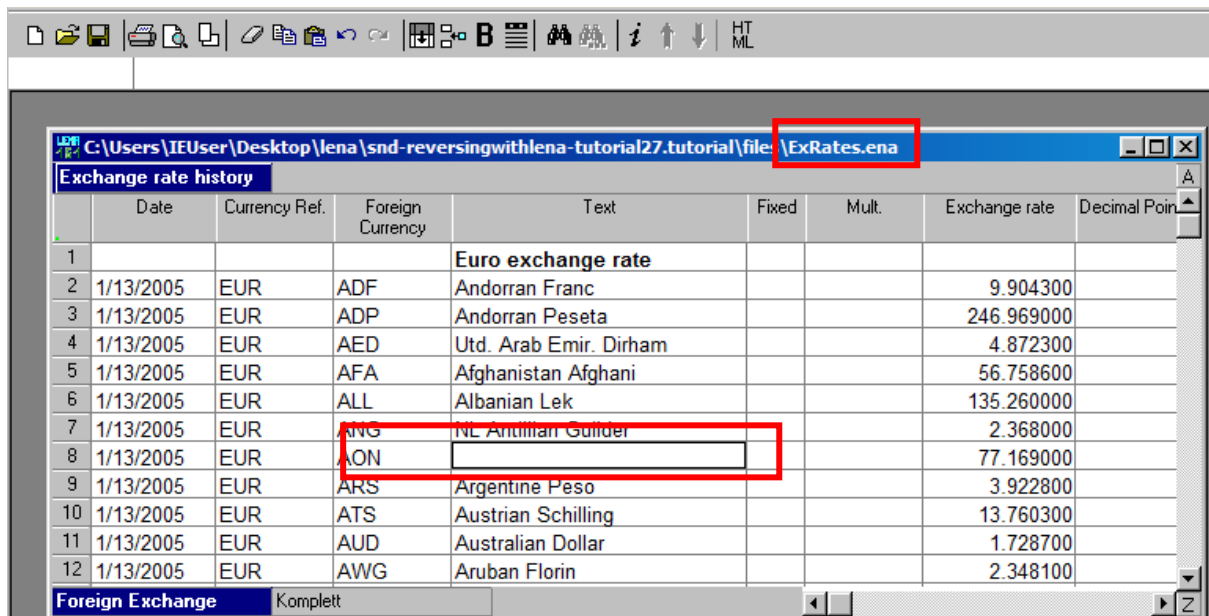


이렇게 깨끗하게 만들어진 상태에서 Fix Dump를 해준다. (덤프 파일된 상태로 저장)

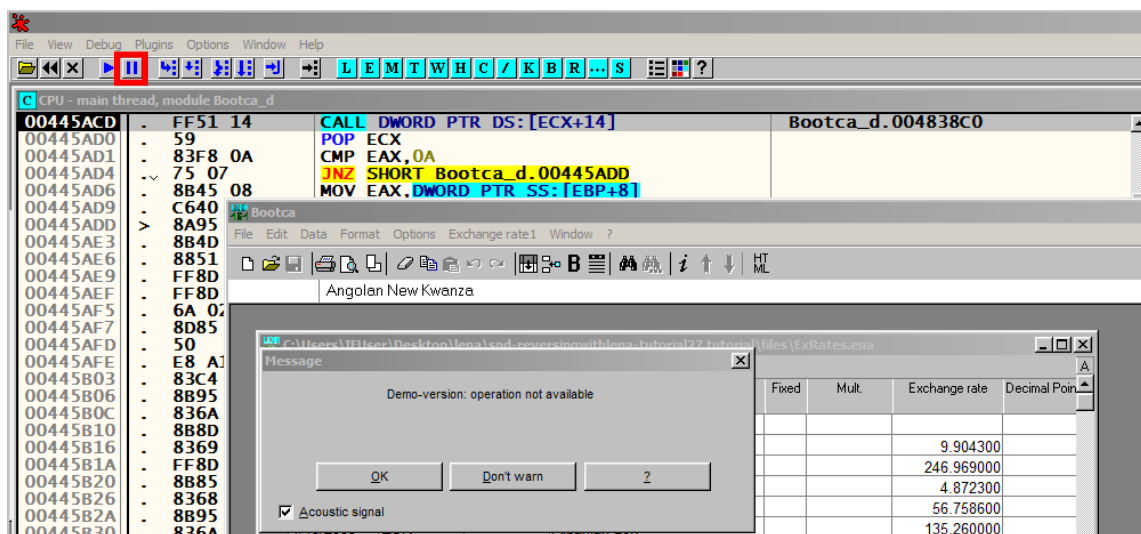


수정 된 덤프 파일을 디버거에서 열어서 실행시키니 정상적으로 실행되고 언패킹된걸 볼 수 있다. 이제 해당 프로그램을 분석할 수 있게 되었다.

2. Bootca.exe – 라이선스 등록



Bootca에서 ExRates.ena를 열고 아무거나 삭제한 다음 저장을 하면 데모버전이라서 이용이 불가능하다고 나온다. 해당 창이 안나오게 키 우회를 해서 사용가능하게 등록해줘야 한다.



이렇게 창이 나온 상태에서 올리디버그로 돌아가 'F12'또는 일시정지 버튼을 눌러준다.

Call stack of main thread				
Address	Stack	Procedure / arguments	Called from	Frame
0012F44C	75926699	Includes ntdll.KiFastSystemCallRet	USER32.75926697	0012F480
0012F450	759437FA	USER32.WaitMessage	USER32.759437F5	0012F480
0012F484	75943AF7	USER32.75943712	USER32.75943AF2	0012F480
0012F4A8	75943B46	USER32.75943A45	USER32.75943B41	0012F4A4
0012F4C8	7595D014	USER32.DialogBoxIndirectParamAorW	USER32.7595D00E	0012F4C4
0012F4F4	00483981	<JMP.&user32.#1680>	Bootca_d.0048397C	0012F4F0
0012F4F8	00400000	hInst = 00400000		
0012F4FC	00000FAF	pTemplate = FAF		
0012F500	0002024A	hOwner = 0002024A (class='MDICliet		
0012F504	004836E0	DlgProc = Bootca_d.004836E0		
0012F508	00000000	lParam = NULL		
0012F510	00483902	Maybe Bootca_d.00483920	Bootca_d.004838FF	0012F50C
0012F514	0012F560	Arg1 = 0012F560		
0012F524	00445AD0	Bootca_d.004838C0	Bootca_d.00445ACD	0012F520
0012F528	0012F560	Arg1 = 0012F560		
0012F62C	0044588E	Bootca_d.00445974	Bootca_d.00445889	0012F628
0012F630	014D5978	Arg1 = 014D5978		
0012F634	00000000	Arg2 = 00000000		
0012F638	014D59B4	Arg3 = 014D59B4		
0012F690	0044562A	Bootca_d.0044564C	Bootca_d.00445625	0012F68C
0012F694	014D5978	Arg1 = 014D5978		
0012F698	00000000	Arg2 = 00000000		
0012F69C	00629C18	Arg3 = 00629C18		
0012F6A0	014DCCE0	Arg4 = 014DCCE0		
0012F6A4	005F0180	Arg5 = 005F0180		

그 상태에서 콜스택을 눌러서 들어가보면 USER32.DialogBoxIndirectParamAorW가 오류 메시지를 보내는 함수라는 걸 알 수 있다.

하지만 <JMP.&USER32.#1680> 이전 함수부터 분석을 해야한다. 그래서 밑에 0x004838FF부터 들어가본다.

※ USER32.DialogBoxIndirectParamAorW 와 <JMP.&USER32.#1680> 차이

- USER32.DialogBoxIndirectParamAorW: 에러 메시지/nag 창을 띄우는 진짜 함수 (실제 일을 하는 놈)

- <JMP.&USER32.#1680>: 이게 그 API를 가리키는 IAT 점프 스텝

해당 부분 들어가보면 JMP dword ptr [&USER32.#1680] 이런식으로 여러 개가 나와 있음

※ 콜스택은 위가 최근에 생성된거 아래가 처음에 생성된거라서 위 -> 아래로 읽어야 한다.

004838DB	84C0	TEST AL,AL
004838DD	75 0F	JNZ SHORT Bootca_d.004838EE
004838DF	68 ED7F0000	PUSH 7FED
004838E4	FF33	PUSH DWORD PTR DS:[EBX]
004838E6	E8 B980FFFF	CALL Bootca_d.0047B9A4
004838EB	83C4 08	ADD ESP,8
004838EE	891D ACAC6000	MOV DWORD PTR DS:[60ACAC],EBX
004838F4	53	PUSH EBX
004838F5	E8 A2FEFFFF	CALL Bootca_d.0048379C
004838FA	59	POP ECX
004838FB	53	PUSH EBX
004838FC	8B43 0C	MOV EAX,DWORD PTR DS:[EBX+C]
004838FF	FF50 18	CALL DWORD PTR DS:[EAX+18]
00483902	59	POP ECX
00483903	8BF0	MOV ESI,EAX

위에 분기문이 있는데 해당 분기문은 구역이 짧아서 0x004838FF에 영향을 주지 않는다.

그래서 이전 함수를 봐줘야한다.

00445A76	64 A3 00000000	MOV DWORD PTR ES:[0],EAX
00445A7C	E9 52010000	JMP Bootca_d.00445BD3
00445A81	66 C785 24FF	MOV WORD PTR SS:[EBP-DC],8
00445A8A	FF75 10	PUSH DWORD PTR SS:[EBP+10]
00445A8D	FFB5 10FFFFFF	PUSH DWORD PTR SS:[EBP-F0]
00445A93	6A 00	PUSH 0
00445A95	8D95 38FFFFFF	LEA EDX,DWORD PTR SS:[EBP-C8]
00445A9B	52	PUSH EDX
00445A9C	E8 DB10FFFF	CALL Bootca_d.00436B7C
00445AA1	83C4 10	ADD ESP,10
00445AA4	8385 30FFFFFF	ADD DWORD PTR SS:[EBP-D0],6
00445AAB	66 C785 24FF	MOV WORD PTR SS:[EBP-DC],14
00445AB4	8B4D 08	MOV ECX,DWORD PTR SS:[EBP+8]
00445AB7	8A41 4C	MOV AL,BYTE PTR DS:[ECX+4C]
00445ABA	8885 64FFFFFF	MOV BYTE PTR SS:[EBP-9C],AL
00445AC0	8D95 38FFFFFF	LEA EDX,DWORD PTR SS:[EBP-C8]
00445AC6	52	PUSH EDX
00445AC7	8B8D 44FFFFFF	MOV ECX,DWORD PTR SS:[EBP-BC]
00445ACD	FF51 14	CALL DWORD PTR DS:[ECX+14]
00445AD0	59	POP ECX
00445AD1	83F8 0A	CMP EAX,0A

0x0045ACD에서 위를 보면 JMP가 있는 걸 볼 수 있는데 해당 분기문이 저장을 할지 안 할지 영향을 주는 부분이라고 생각할 수 있다.

00445A4D	8078 4E 00	CMP BYTE PTR DS:[EAX+4E],0
00445A51	74 14	JE SHORT Bootca_d.00445A67
00445A53	8B95 14FFFFFF	MOV EDX,DWORD PTR SS:[EBP-EC]
00445A59	64 8915 0000	MOV DWORD PTR FS:[0],EDX
00445A60	8BC2	MOV EAX,EDX
00445A62	E9 6C010000	JMP Bootca_d.00445BD3
00445A67	8B4D 08	MOV ECX,DWORD PTR SS:[EBP+8]
00445A6A	8079 4D 00	CMP BYTE PTR DS:[ECX+4D],0
00445A6E	75 11	JNZ SHORT Bootca_d.00445A81
00445A70	8B85 14FFFFFF	MOV EAX,DWORD PTR SS:[EBP-EC]
00445A76	64 A3 00000000	MOV DWORD PTR FS:[0],EAX
00445A7C	E9 52010000	JMP Bootca_d.00445BD3
00445A81	66 C785 24FF	MOV WORD PTR SS:[EBP-DC],8
00445A8A	FF75 10	PUSH DWORD PTR SS:[EBP+10]
00445A8D	FFB5 10FFFFFF	PUSH DWORD PTR SS:[EBP-F0]
00445A93	6A 00	PUSH 0
00445A95	8D95 38FFFFFF	LEA EDX,DWORD PTR SS:[EBP-C8]
00445A9B	52	PUSH EDX

하지만 위에 좀 더 올라가보면 JNZ 분기문이 있는 걸 볼 수 있고 해당 분기문이 JMP에 영향을 주는 걸 볼 수 있다. 하지만 좀 더 올라가보면 JE랑 다른 JMP가 있는 걸 볼 수 있는데 전 부다 영향을 주는 걸 볼 수 있다.

00445A4D	. 8078 4E 00	CMP BYTE PTR DS:[EAX+4E],0
00445A51	. 74 14	JE SHORT Bootca_d.00445A67
00445A53	. 8B95 14FFFFFF	MOV EDX,DWORD PTR SS:[EBP-EC]
00445A59	. 64:8915 0000	MOV DWORD PTR FS:[0],EDX
00445A60	. 8BC2	MOV EAX,EDX
00445A62	. E9 6C010000	JMP Bootca_d.00445BD3
00445A67	. 8B4D 08	MOV ECX,DWORD PTR SS:[EBP+8]
00445A6A	. 8079 4D 00	CMP BYTE PTR DS:[ECX+4D],0
00445A6E	. 75 11	JNZ SHORT Bootca_d.00445A81
00445A70	. 8B85 14FFFFFF	MOV EAX,DWORD PTR SS:[EBP-EC]
00445A76	. 64:A3 000000	MOV DWORD PTR FS:[0],EAX
00445A7C	. E9 52010000	JMP Bootca_d.00445BD3
00445A81	. 66:C785 24FF	MOV WORD PTR SS:[EBP-DC],8
00445A8A	. FF75 10	PUSH DWORD PTR SS:[EBP+10]
00445A8D	. FF85 10FFFFFF	PUSH DWORD PTR SS:[EBP-E0]

확실히 판단하기 위해서 테스트를 해봐야한다. 0x00445A51에 BP를 걸고 다시 실행해보면 해당 부분에서 멈추는데 원래 작동되던 방식과 다르게 해야한다. 점프를 안하게 바꿔주고 'F9'를 눌러 실행해보면 nag창은 뜨진 않지만 저장이 되지 않아서 저장과 상관없는 부분이라는 걸 알 수 있다.

그럼 다시 이전 함수를 분석해야하므로 이전 함수로 들어가준다.

00445829	. 74 36	JE SHORT Bootca_d.00445861
0044582B	. 83C4 F8	ADD ESP,-8
0044582E	. 66:C745 C0 80	MOV WORD PTR SS:[EBP-40],8C
00445834	. 8B55 08	MOV EDX,DWORD PTR SS:[EBP+8]
00445837	. 83C2 3C	ADD EDX,3C
0044583A	. 52	PUSH EDX
0044583B	. 8D4C24 04	LEA ECX,DWORD PTR SS:[ESP+4]
0044583F	. 51	PUSH ECX
00445840	. E8 D36D0B00	CALL Bootca_d.004FC618
00445845	. 83C4 08	ADD ESP,8
00445848	. FF45 CC	INC DWORD PTR SS:[EBP-34]
0044584B	. 8B45 08	MOV EAX,DWORD PTR SS:[EBP+8]
0044584E	. FF30	PUSH DWORD PTR DS:[EAX]
00445850	. 66:C745 C0 80	MOV WORD PTR SS:[EBP-40],80
00445856	. FF4D CC	DEC DWORD PTR SS:[EBP-34]
00445859	. E8 CA91FCFF	CALL Bootca_d.0040EA28
0044585E	. 83C4 0C	ADD ESP,0C
00445861	. FF4D CC	DEC DWORD PTR SS:[EBP-34]
00445864	. 836D CC 02	SUB DWORD PTR SS:[EBP-34],2
00445868	. 6A 00	PUSH 0
0044586A	. 8D55 D8	LEA EDX,DWORD PTR SS:[EBP-28]
0044586D	. 52	PUSH EDX
0044586E	. E8 0D9D0C00	CALL Bootca_d.0050F580
00445873	. 83C4 08	ADD ESP,8
00445876	. 66:C745 C0 00	MOV WORD PTR SS:[EBP-40],8
0044587C	. 8B4D 08	MOV ECX,DWORD PTR SS:[EBP+8]
0044587F	. 83C1 3C	ADD ECX,3C
00445882	. 51	PUSH ECX
00445883	. FF75 0C	PUSH DWORD PTR SS:[EBP+C]
00445886	. FF75 08	PUSH DWORD PTR SS:[EBP+8]
00445889	. E8 E6000000	CALL Bootca_d.00445974
0044588E	. 83C4 0C	ADD ESP,0C

위로 올라가면 JE분기문을 찾을 수 있는데 분기문의 구역이 짧으므로 해당 분기문은 함수에 영향을 주지 않는 걸 볼 수 있다.

004455C0	\$ 55	PUSH EBP
004455C1	8BEC	MOV EBP,ESP
004455C3	83C4 DC	ADD ESP,-24
004455C6	B8 24C75F00	MOV EAX,Bootca_d.005FC724
004455CB	E8 D0141700	CALL Bootca_d.005B6AA0
004455D0	C745 F8 0100	MOV DWORD PTR SS:[EBP-8],1
004455D7	66:C745 EC 08	MOV WORD PTR SS:[EBP-14],8
004455DD	83C4 F4	ADD ESP,-0C
004455E0	66:C745 EC 20	MOV WORD PTR SS:[EBP-14],20
004455E6	8B55 18	MOV EDX,DWORD PTR SS:[EBP+18]
004455E9	895424 04	MOV DWORD PTR SS:[ESP+4],EDX
004455ED	8B4D 1C	MOV ECX,DWORD PTR SS:[EBP+1C]
004455F0	894C24 08	MOV DWORD PTR SS:[ESP+8],ECX
004455F4	B8 80015F00	MOV EAX,Bootca_d.005F0180
004455F9	890424	MOV DWORD PTR SS:[ESP],EAX
004455FC	FF45 F8	INC DWORD PTR SS:[EBP-8]
004455FF	83C4 F8	ADD ESP,-8
00445602	FF75 10	PUSH DWORD PTR SS:[EBP+10]
00445605	8D5424 04	LEA EDX,DWORD PTR SS:[ESP+4]
00445609	52	PUSH EDX
0044560A	E8 01C50B00	CALL Bootca_d.00501B10
0044560F	83C4 08	ADD ESP,8
00445612	FF45 F8	INC DWORD PTR SS:[EBP-8]
00445615	FF75 0C	PUSH DWORD PTR SS:[EBP+C]
00445618	FF75 08	PUSH DWORD PTR SS:[EBP+8]
0044561B	66:C745 EC 14	MOV WORD PTR SS:[EBP-14],14
00445621	8345 F8 FE	ADD DWORD PTR SS:[EBP-8],-2
00445625	E8 22000000	CALL Bootca_d.0044564C
0044562A	83C4 1C	ADD ESP,1C

다시 이전 함수를 분석해야하므로 들어가 본다. 해당 함수가 있는 부분은 분기문이 없으므로 이전 함수를 분석해야 한다.

0044524F	90	NOP
00445250	\$ 55	PUSH EBP
00445251	8BEC	MOV EBP,ESP
00445253	83C4 DC	ADD ESP,-24
00445256	B8 44C55F00	MOV EAX,Bootca_d.005FC544
0044525B	E8 40181700	CALL Bootca_d.005B6AA0
00445260	83C4 F4	ADD ESP,-0C
00445263	66:C745 EC 14	MOV WORD PTR SS:[EBP-14],14
00445269	6A FF	PUSH -1
0044526B	6A FF	PUSH -1
0044526D	8D5424 08	LEA EDX,DWORD PTR SS:[ESP+8]
00445271	52	PUSH EDX
00445272	E8 F1700B00	CALL Bootca_d.004FC368
00445277	83C4 0C	ADD ESP,0C
0044527A	FF45 F8	INC DWORD PTR SS:[EBP-8]
0044527D	FF75 0C	PUSH DWORD PTR SS:[EBP+C]
00445280	6A 00	PUSH 0
00445282	FF75 08	PUSH DWORD PTR SS:[EBP+8]
00445285	66:C745 EC 08	MOV WORD PTR SS:[EBP-14],8
0044528B	FF4D F8	DEC DWORD PTR SS:[EBP-8]
0044528E	E8 2D030000	CALL Bootca_d.004455C0
00445293	83C4 18	ADD ESP,18
00445296	8B4D DC	MOV ECX,DWORD PTR SS:[EBP-24]
00445299	6A 8900 0000	MOV DWORD PTR SS:[EBP-10],ECX

해당 함수도 분기문이 없으므로 이전 함수를 분석해야 한다.

0042CB9A	. FF75 08	PUSH DWORD PTR SS:[EBP+8]	[Arg1
0042CB9D	. E8 5A060000	CALL Bootca_d.0042D1FC	Bootca_d.0042D1FC
0042CBA2	. 59	POP ECX	
0042CBA3	. 8AC0	TEST AL, AL	
0042CBA5	. 74 5A	JE SHORT Bootca_d.0042CC01	
0042CBA7	. 6A 00	PUSH 0	[Arg2 = 00000000
0042CBA9	. FF75 08	PUSH DWORD PTR SS:[EBP+8]	Arg1
0042CBAC	. E8 570B0000	CALL Bootca_d.0042D708	Bootca_d.0042D708
0042CBB1	. 83C4 08	ADD ESP, 8	
0042CBB4	. 6A 01	PUSH 1	
0042CBB6	. E8 611EFEFF	CALL Bootca_d.0040EA1C	
0042CBBB	. 50	PUSH EAX	[Arg1
0042CBBE	. E8 53A1FEFF	CALL Bootca_d.00416D14	Bootca_d.00416D14
0042CBC1	. 83C4 08	ADD ESP, 8	
0042CBC4	. 68 FD0B0000	PUSH 0BFD	
0042CBC9	. E8 4E1EFEFF	CALL Bootca_d.0040EA1C	
0042CBCE	. 50	PUSH EAX	[Arg1
0042CBCE	. E8 7C860100	CALL Bootca_d.00445250	Bootca_d.00445250
0042CBD4	. 83C4 08	ADD ESP, 8	
0042CBD7	. B0 01	MOV AL, 1	
0042CBD9	. 50	PUSH EAX	

해당 함수에서 위 코드를 보면 JE분기문이 있는 걸 볼 수 있고 해당 함수에 영향을 미치는 걸 볼 수 있다. 위의 AL에서 점프가 될지 안될지 결정이 되므로 0x0042CB9D의 CALL 함수를 들어가서 분석해준다.

0042D1FC	. \$ 55	PUSH EBP
0042D1FD	. 8BEC	MOV EBP, ESP
0042D1FE	. 8B45 08	MOV EAX, DWORD PTR SS:[EBP+8]
0042D202	. 8A40 24	MOV AL, BYTE PTR DS:[EAX+24]
0042D205	. 5D	POP EBP
0042D206	. C3	RETN

CALL함수로 들어가보면 해당 코드가 나오는 걸 볼 수 있는데 AL이 0x0042D202에서 결정되는 걸 볼 수 있다.

여기에서 코드를 바꾸는 방법이 2가지가 있다.

1. 단순 멍청한 패칭 방법(Plain stupid patching method)

↳ 0x0012CBA5 JE를 항상 점프하도록 패치하는 방식

2. 중간 단계 패칭 방법(Intermediate patching method)

↳ 0x0042D202 MOV AL, BYTE PTR DS:[EAX+24] -> MOV AL, 0으로 바꾸는 방식

3. 고급 패칭 방법(advanced patching method)

↳ AL = 0 으로 바꿔주는 방식

고급 패칭 방식으로 바꿔줄 거다.

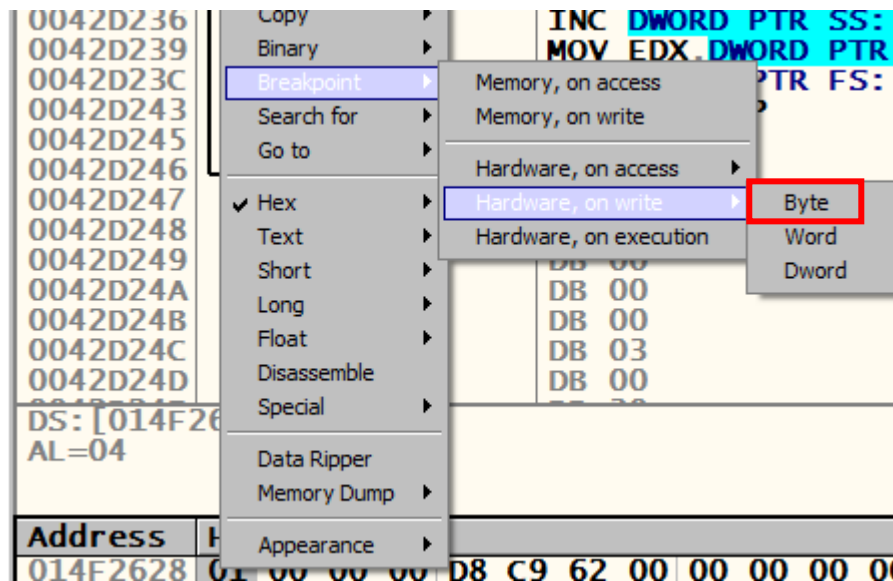
0042D1FC	. \$ 55	PUSH EBP
0042D1FD	. 8BEC	MOV EBP, ESP
0042D1FE	. 8B45 08	MOV EAX, DWORD PTR SS:[EBP+8]
0042D202	. 8A40 24	MOV AL, BYTE PTR DS:[EAX+24]
0042D205	. 5D	POP EBP

DS:[014F2628]=01
AL=04

현재 BYTE PTR DS:[EAX+24] 이 부분은 0x014F2628 위치인 걸 볼 수 있고 01로 설정되어 있는 걸 볼 수 있다. 덤프창에서 해당 위치로 가본다.

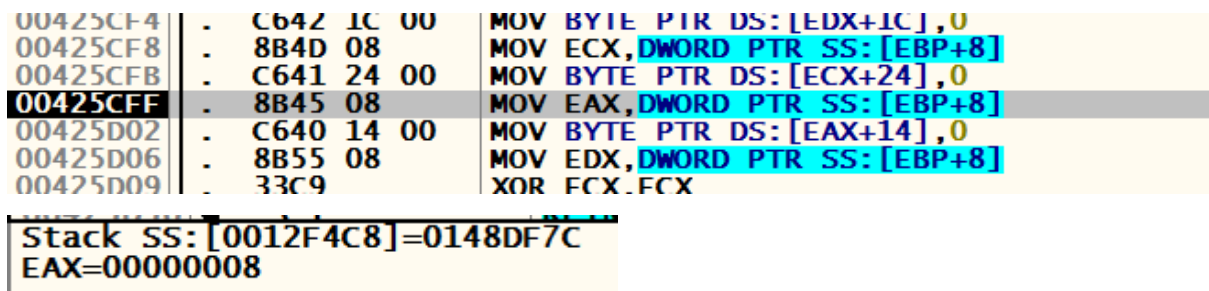


이렇게 01로 되어 있는 걸 볼 수 있고 0x014F2628 위치가 바뀔 때의 코드를 바꿀거다.

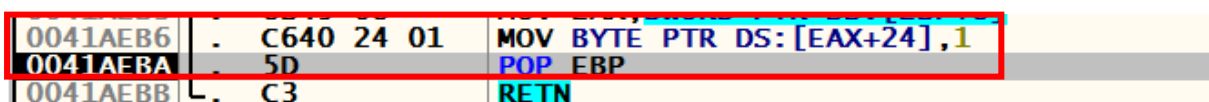


해당 부분이 쓰여질 때 1Byte를 확인하고 싶으니까 Byte로 HWBP를 걸어준다.

프로그램을 재시작해주고 다시 파일을 열어줄 때



이렇게 멈추는 걸 볼 수 있다. 하지만 이 부분은 쓸모가 없는 부분이므로 'F9'를 눌러 다음으로 넘어가준다.



0x0041AEB6에서 멈추고 위에 코드를 보면 EAX = 1로 변하는 부분이 있는 걸 볼 수 있다.

0041AEB3	. 8B45 08	MOV EAX,DWORD PTR SS:[EBP+8]
0041AEB6	. C640 24 00	MOV BYTE PTR DS:[EAX+24],0
0041AEB9	. 5D	POP EBP
0041AEBB	. C3	RETN

MOV BYTE PTR DS:[EAX+24],1 -> MOV BYTE PTR DS:[EAX+24],0으로 바꾸고 저장해준다.

저장한 걸 실행하면 nag창이 안뜨고 저장되는 걸 볼 수 있다.